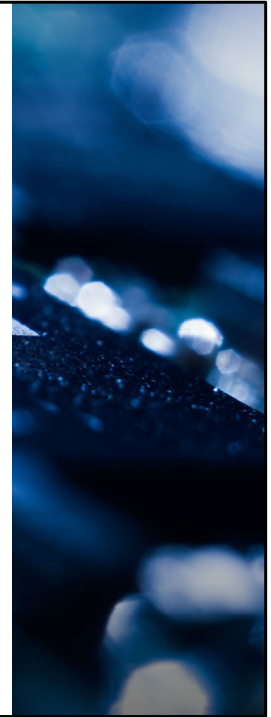




SC1006 Computer Organisation and Architecture

Direct Memory Access

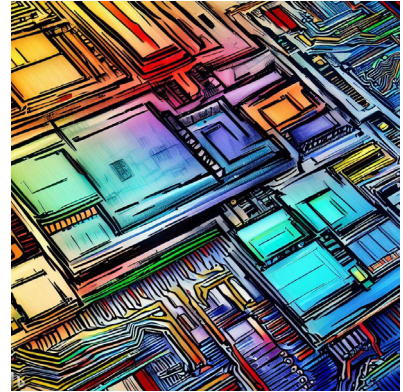
Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



- This section is on Direct Memory Access, DMA in short. We will go through the operation of a DMA controller and how it can be used to relieve the CPU from data transfer task.

Optimizing CPU resources in Data Transfer

- Till now, we have been using CPU to perform the data transfer.
- This becomes increasing inefficient as amount of data increase as CPU has to spend most of its time moving data.
- As such, CPU has less time to perform algorithm processing.
- DMA controller (DMAC) is added to relieve CPU of the data transfer task.
- DMAC has dedicated hardware that could move data more efficiently than CPU in scenarios where complex address manipulation is required. E.g. de-interleaving Left and Right Channel Audio data.
- If there are no conflicts in hardware resources used, it is possible for data transfer via DMA and CPU execution to occur simultaneously.
- If there is a conflict in hardware resources used, e.g. both DMAC and the CPU needs the system bus, then access will be given to the one with higher priority.
- Who has the higher priority and whether the priority is configurable depends on processor design and is thus processor specific.



Up till this point, our discussion on data transfer mechanism has always involve CPU as the controller that performs the data transfer task.

As the amount of data increases, the time needed to transfer data increases as well. So it would appear that we are not using the CPU effectively as the main purpose of a CPU should be data manipulations rather than transferring data from point A to point B.

If we are able to relieve the processor from the data transfer task and have it focus on algorithm processing, the overall system will be better utilised and efficient. This is where the DMA controller comes in.

DMA Controller is a controller that is able to control the system bus to perform data transfer without help from CPU.

In doing so, it relieves the CPU from doing data transfer so that it could focus on processing data.

In addition, a DMA controller has dedicated hardware that if well designed, would be more efficient than CPU when transferring data which requires complex address manipulation.

For example, de-interleaving the Left and Right channel of audio data.

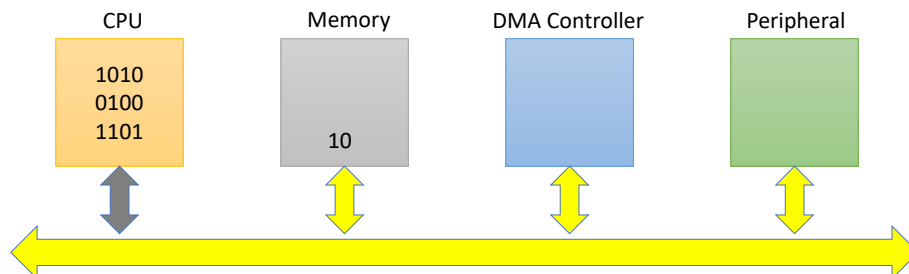
Operation wise, it is possible to have both the CPU execution and DMAC data transfer to happen at the same time if both of them are not using any common resources.

But if they are using the same resource, e.g. system bus, then one of them will have to wait.

With common resources, there will be a need for arbitration, and one way of doing it is via priority. Whoever has the higher priority will get to use the system bus. In real word processor, this priority can be fixed, either CPU has higher priority compared to DMAC, or vice versa. Or it could very well be configurable.

For our course, we will assume that there is only one system bus available in the processor. That means the CPU and DMA will have to fight of the usage of the system bus.

DMA Controller (DMAC)



- DMAC is a **Data Bus Controller** module that performs data transfer independent of CPU, between **Memory and memory, I/O Peripheral and I/O peripheral, Memory and I/O Peripheral**.
- Generates address and initiates read/write operation between devices mentioned above.
- Note that 'Peripheral' here could be **internal or external** peripherals
- The DMAC shown here uses the **Fetch and Deposit** DMA mechanism where the data goes into the internal buffer of the DMAC before being transferred to the Destination.

A DMA controller, DMAC in short, can transfer data between various modules within a computer system.

It does this by generating the addresses and initiate the read/write controls. The DMAC shown here is known as a Fetch-and-Deposit type of DMA, where the data from the source will be transferred into the DMAC before it is transported to the destination.

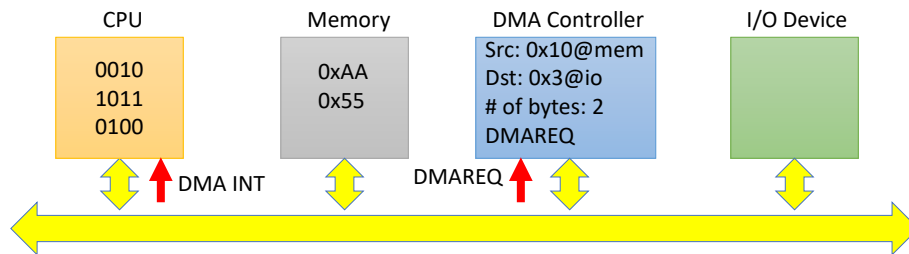
The following animation illustrates how bus is shared between CPU and DMAC.

The CPU may be doing some operations that doesn't require the system bus. So DMAC can perform data transfer between the memory and IO device. As and when the CPU needs the system bus, control will be given to the CPU. And after the CPU completes its data transfer, the DMAC can continue its operation.

In real world design, the bus priority between the CPU and DMAC may differ from the illustration.

In this example, we are assuming that CPU has a higher priority than DMAC.

Basic DMA Process



- DMA Configuration Parameters initialization (by CPU to DMAC)
 - Source Address
 - Destination Address
 - Amount of data to transfer
 - DMA Trigger signal (DMAREQ, interrupt, etc)
- CPU proceed with its own task while DMAC wait for DMA Trigger Signal.
- Once DMA Trigger occurs, DMAC request to take over system bus.
- DMAC transfer data according to configuration.
- DMAC notify CPU of data completion (typically via interrupts) and release system bus.

Now that we have some idea of what a DMA is, let's walk thru a DMA process to understand

how a DMAC is configured and the various control signals involved.

In order for DMAC to transfer data, it needs a few key parameters. Typically, these parameters are configured by the CPU to the DMAC.

Source Address of Data

Destination Address of Data

Number of words of Data to transfer

And the trigger signal to tell DMAC when to start the transfer

After the DMAC is configured, it'll just wait in IDLE state for the trigger signal.

Once the trigger signal comes in, it'll request for the system bus and start the data transfer.

When all the data has been transferred, DMAC will release the system bus and issue a DMA interrupt signal to the CPU, informing CPU that the data is ready to be processed.

DMA Mode of Operation

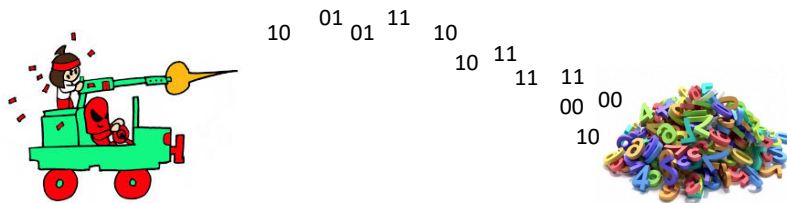
- Different processors has [different DMAC design](#), typically with slight variance in terms of how they transfer data and the data type (Audio, Video etc) they are optimized for.
- In general, DMAC could transfer the block of data they are tasked to transfer in the following mode
 - [Burst](#)
 - [Cycle Stealing](#)
 - [Transparent](#)
- Do note that you may encounter DMAC design that varies from the basic mode of operation discussed here but general concept will still apply.
- In fact, for majority of the time, you'll encounter DMAC design that uses [a combination of the modes](#) described



- Different DMACs may differ in the way they transfer data.
- There are three basic data transfer modes that DMAC typically use, namely, Burst, Cycle Stealing and Transparent. We will get to the details of each mode in later slides.
- Note, however that in an actual processor, it is likely for a DMAC to use a combination of these modes.

Burst Mode

- The DMA controller gains control of the system bus and **transfer multiple units of data** before returning control of the bus to the CPU.
- Note that the burst could be **the entire block of data** DMAC is tasked to transfer, or a **subset of the block**, i.e. it may take a few burst to complete the transfer of the entire block.
- CPU may continue to operate as long as it does not need access the particular system bus that DMAC is using.
- If CPU needs to access the system bus, **CPU may be suspended** till DMAC has completed its data transfer.
- Fast data transfer rate but may render the CPU inactive for **longer period** of time.



First, let's look at the Burst Mode.

In Burst Mode, DMAC will transfer multiple units of data when they take control of the system bus.

This could be the entire block of data the DMAC is tasked to transfer, OR
It could be a subset of the block, that means it probably take the DMAC a few bursts to complete the block transfer.

CPU can continue with its operation as long as there are no conflicts in resource utilisation, for example the system bus.

However, if the CPU needs system bus during the time when DMAC is doing the transfer, the CPU may need to wait till DMAC complete its transfer.

Burst mode would result in faster data transfer rate but may potentially cause the CPU to be suspended for extended period.

Cycle Stealing

- DMAC releases the system bus after transferring **one unit of data**.
- Depending on processor design, DMAC tends to execute the data transfer
 - **Between CPU instructions**
 - **Between Pipeline stages**
- CPU may be suspended if it need to access the system bus but **suspend time is shorter** as only one unit is transferred at one time.
- Transfer rate is **slower** than in Burst Mode but will CPU will only be **inactive for very short period of time**.
- **Favoured** in application which requires CPU to be **responsive** e.g. real-time security status monitoring.



Between
CPU Instr

CPU	Instr1	Instr2		Instr3		Instr4
DMAC			Data		Data	

Between
Pipeline
Stages

CPU	Fetch Instr	Decode Instr		Fetch Operand	Exe Instr	
DMAC			Data			Data

If DMAC is using Cycle Stealing mode, it will only transfer one unit of data after taking control of the system bus, after which it will release the bus back to the CPU.

The name cycle stealing comes about as the DMAC will try to adjust its use of system bus between processor operations.

For example, it will tend to execute the data transfer
Between CPU instructions, OR
Between Processor Pipeline stages

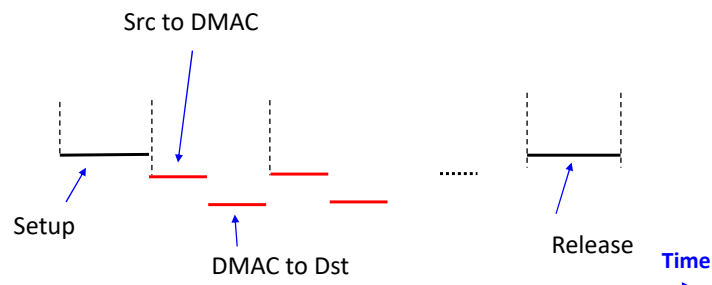
Any resultant suspension of CPU execution is kept short because only one unit of data will be transferred so CPU will be more responsive to other events.

The time available for data transfer is shorter than that of the Burst Mode; therefore, data transfer rate will be lower as well.

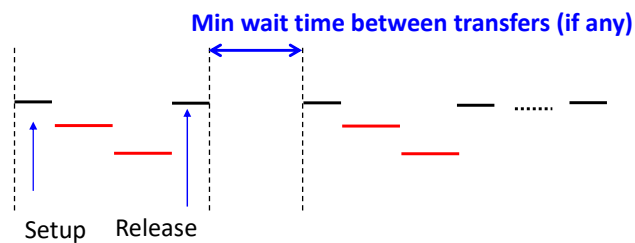
As this method is less disruptive to CPU operation compared to Burst Mode, it is preferred in systems that require CPU to be more responsive, for example, to achieve real-time monitoring.

Fetch and Deposit (Timings)

Burst



Cycle Stealing



This is a slide to illustrate, from timing diagram perspective, the DMA operation for the burst and the cycle stealing modes.

In the burst mode

For for the burst ,

Recall that once the DMAC have the control for the system bus, it'll transfer the entire block of data to the destination.

Timing wise

Setup correspond to the time DMAC takes to request control of the system bus from the CPU

This is followed by transfer of data

Each unit of data is transfer in two stages:

first from Source to DMAC internal buffer

Second from DMAC to the destination.

After the last unit of data is transferred to the destination, the DMAC will release the bus control back to the CPU.

In Cycle Stealing mode

The DMAC will request control of the system bus from the CPU, transfer one unit of data and release the bus back to the CPU.

The DMAC may also be required to wait for a certain time duration before it can request for the bus control again.

The timing is as shown in the diagram

The DMAC will request for the bus control, which is 'Setup' in the diagram, it will then transfer the data, release the bus back to CPU and wait for a fixed duration before requesting for the bus again.

Take note of these timing diagrams as you will need them to complete your tutorial questions.

Transparent Mode



- DMAC transfer data only when **CPU is not using the data bus**.
- **Zero impact** to CPU performance in terms of data bus access.
- Potentially the slowest transfer rate among the three modes.
- **More Complex hardware** needed to detect when CPU is not using the data bus.
- The CPU activity detection technique could also be used by some processors as a cue to trigger a burst transfer only when CPU is not using the system bus.

- Lastly is the Transparent Mode, which can be viewed as a special case of cycle stealing.
- In this mode, the DMAC will only start a transfer if it detects that the system bus is not being used by the CPU.
- This results in the least disruption to the CPU but also has the slowest data transfer rate among the three DMA transfer modes.
- In addition, it would require some complex hardware that allows the DMAC to determine the activity on the bus.

Comparison of DMA Transfer Modes

- **Burst**

- Allow **faster** data transfer
- CPU may be **suspended** for **longer** period of time
- May not be suitable for Real-time applications with CPU
- Suitable for application where transfer is bursty e.g. HDD file transfer.

- **Cycle Stealing**

- **Interleaving** DMA data transfer with CPU instructions allows CPU to continue executing its program while DMAC is doing data transfer.
- **CPU performance lower** due to interleaving of DMA transfers, but would be **more responsive** than in Burst mode.
- **Slower data transfer rate** than Burst mode.
- Suitable for Real-time applications with CPU

- **Transparent**

- Potentially would not affect CPU performance at all.
- **Slowest data transfer rate** but best CPU response
- **More complex hardware** needed to detect when CPU is not using the bus.



To summarise.

Burst Mode allows a high transfer rate but may potentially suspend the CPU for an extended period of time.

Suitable for applications where data transfer is bursty in nature, such as HDD transfer.

Cycle Stealing mode interleaves its data transfer with CPU instructions so it has less impact to the CPU performance.

This allows the CPU to be more responsive, but at a trade off of lower DMA transfer rate.

Transparent mode of DMA does not affect the CPU performance at all as it only work when CPU is not using the system bus.

However, implementation may be more complex as it now has to predict/detect when the system bus is free.

The rate of data transfer becomes very variable.